HIGH-LEVEL DESIGN

RAG-Based Customer Support Assistant

Document Type	High-Level Design (HLD)
Project	RAG Customer Support System with LangGraph & HITL
Version	1.0
Status	Final

TABLE OF CONTENTS

1. SYSTEM OVERVIEW

1.1 Problem Definition

Modern customer support teams are overwhelmed by repetitive queries that could be answered directly from product documentation, FAQs, or policy PDFs. Traditional keyword-search systems lack contextual understanding, causing high escalation rates and slow resolution times. A Retrieval-Augmented Generation (RAG) system bridges this gap by combining semantic search over a curated knowledge base with a powerful language model, enabling contextually accurate, grounded responses.

1.2 Scope

In Scope	PDF knowledge-base ingestion • Semantic chunking & embedding • ChromaDB vector storage • Query intent classification • LLM-powered answer generation • LangGraph workflow orchestration • HITL escalation for low-confidence / complex queries
Out of Scope	Real-time web crawling • Multi-tenant isolation • Fine-tuning the base LLM • Mobile app development

1.3 Goals & Non-Goals

- Provide accurate, source-grounded answers from PDF knowledge bases.
- Reduce mean resolution time for Tier-1 support queries by ≥ 60%.
- Escalate ambiguous or sensitive queries to a human agent seamlessly.
- Maintain full auditability of every query-response pair.

2. ARCHITECTURE DIAGRAM

The diagram below illustrates the end-to-end system. Data enters through the User Interface, is processed by the LangGraph Workflow Engine, retrieves context from ChromaDB, generates an answer via the LLM, and optionally escalates to a human agent through the HITL module.

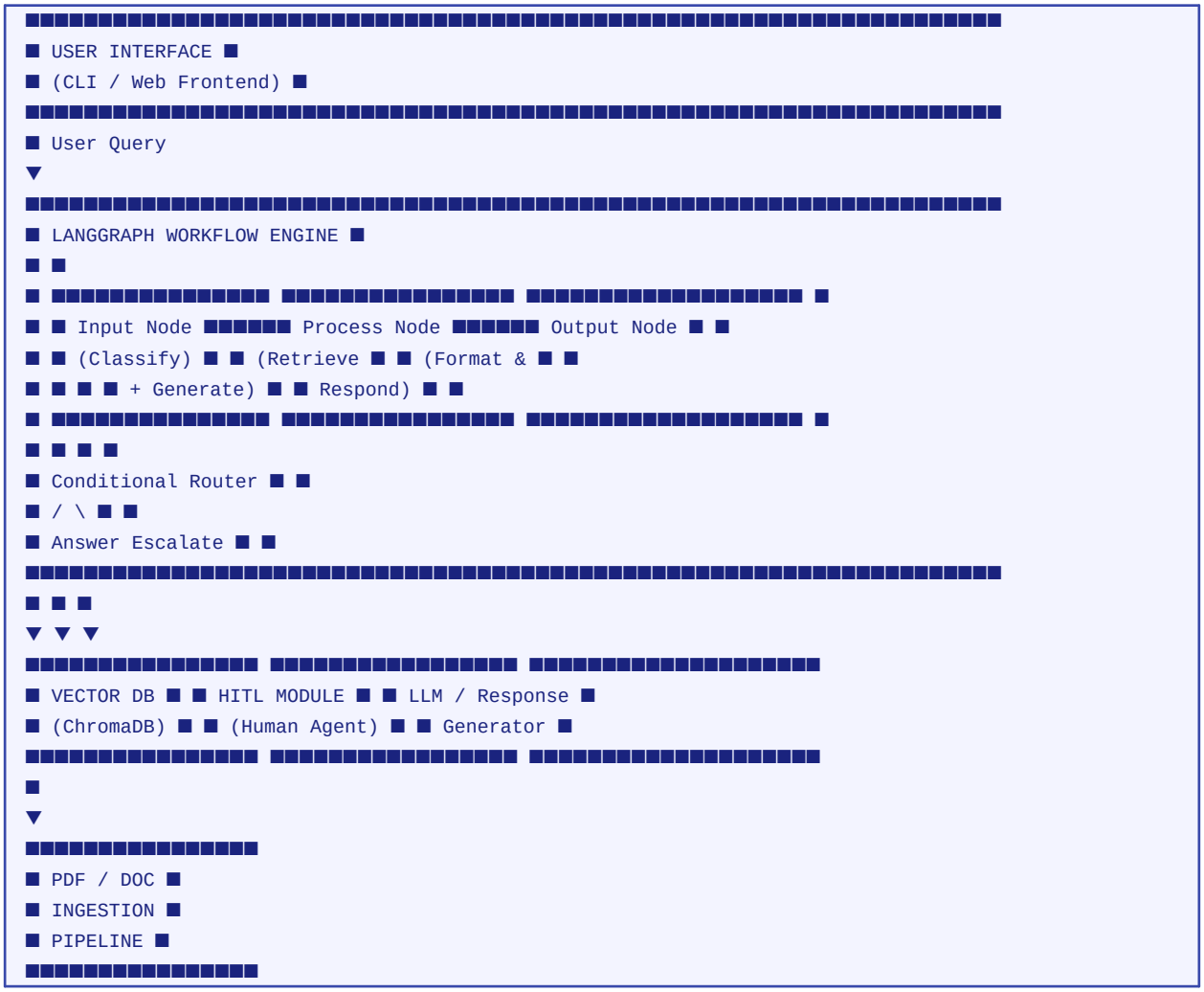

Figure 1 – High-Level System Architecture

2.1 Key Architectural Layers

Layer	Responsibility	Key Components
Presentation	Accept user input; render responses	CLI / Web UI
Orchestration	Control query lifecycle via a state machine	LangGraph Engine
Retrieval	Semantic search over embedded knowledge	ChromaDB, Embedder
Generation	Produce grounded natural-language answers	LLM (GPT-4o / Llama)
Escalation	Route complex queries to human agents	HITL Module
Ingestion	Process and index source documents	PDF Loader, Chunker

3. COMPONENT DESCRIPTIONS

■ Document Loader

Reads PDF files from disk using LangChain's PyPDFLoader. Handles multi-page documents, extracts raw text per page, and passes the page stream to the Chunking Module.

■ Chunking Module

Applies RecursiveCharacterTextSplitter with chunk_size=500 and chunk_overlap=50. Overlap preserves sentence context across chunk boundaries, reducing the risk of losing key information mid-sentence.

■ Embedding Module

Converts text chunks into dense 1536-dimensional vectors using OpenAI text-embedding-3-small (or a local SentenceTransformer model for cost-free inference). Embeddings capture semantic meaning rather than exact keywords.

■ Vector Store (ChromaDB)

Persists chunk embeddings in a local ChromaDB collection. Supports cosine similarity search. Chosen for zero-configuration local deployment and native LangChain integration.

■ Retriever

Queries ChromaDB for the top-k (k=4) most semantically similar chunks given the user's query embedding. Returns ranked context snippets with source metadata.

■ LLM Layer

A prompted language model (GPT-4o or Llama-3) receives the user query plus retrieved context and generates a grounded answer. The prompt instructs the model to answer ONLY from provided context and flag gaps.

■ LangGraph Workflow Engine

Orchestrates the query lifecycle as a stateful directed graph. Nodes represent discrete processing steps; edges encode transitions. A conditional router decides: answer, escalate, or clarify.

■ Routing Layer

Evaluates retrieval confidence (similarity scores), query complexity heuristics, and LLM self-reported uncertainty to choose the appropriate output branch.

■ HITL Module

When escalation is triggered, the query is placed in a human-review queue. An agent provides a response which is injected back into the graph state and returned to the user. All interactions are logged.

4. DATA FLOW

4.1 Document Ingestion Pipeline

Step 1 – Load	PDF file read by PyPDFLoader → raw text per page
Step 2 – Chunk	RecursiveCharacterTextSplitter splits pages into ~500-char chunks
Step 3 – Embed	Each chunk converted to a 1536-d vector via Embedding Model
Step 4 – Store	Vectors + metadata (source, page_no) persisted in ChromaDB

4.2 Query Lifecycle

Q1 – Receive	User submits natural-language query through UI
Q2 – Classify	Input Node detects intent: FAQ / complaint / out-of-scope
Q3 – Embed	Query converted to embedding vector
Q4 – Retrieve	Top-4 chunks fetched from ChromaDB by cosine similarity
Q5 – Generate	LLM constructs grounded answer from query + context
Q6 – Route	Confidence evaluated → answer branch or escalation branch
Q7 – Respond	Final answer or HITL response returned to user

5. TECHNOLOGY CHOICES

Technology	Role	Rationale
LangChain	Document loading & chaining	Mature ecosystem; native ChromaDB + LLM integrations
ChromaDB	Vector store	Zero-config local setup; cosine similarity built-in; LangChain-native
LangGraph	Workflow orchestration	Stateful graph model; first-class conditional routing; HITL support
OpenAI / Llama-3	Language model	GPT-4o for quality; Llama-3 for cost/privacy trade-off
text-embedding-3-small	Embedding model	High quality at low cost; 1536 dimensions; OpenAI API
FastAPI (optional)	REST interface	Lightweight async Python web framework
Python 3.11	Runtime	Rich ML/NLP ecosystem; async support

6. SCALABILITY CONSIDERATIONS

6.1 Handling Large Documents

- Hierarchical chunking: coarse chunks for broad retrieval, fine chunks for precision.
- Async ingestion pipeline with batch embedding API calls to avoid rate limits.
- ChromaDB can be replaced with Weaviate or Pinecone for distributed, million-scale collections.
- Source documents partitioned by domain (e.g., billing, shipping) for filtered retrieval.

6.2 Increasing Query Load

- LangGraph workflow is stateless per request — horizontally scalable behind a load balancer.
- Embedding cache (Redis) for repeated queries reduces LLM/embedding API calls.
- ChromaDB read replicas for concurrent vector searches.
- Async FastAPI endpoint handles hundreds of concurrent requests.

6.3 Latency Optimisation

- Target P95 latency < 2 s by caching embeddings of the 500 most-frequent queries.
- Streaming LLM responses (Server-Sent Events) improve perceived latency.
- Retrieval pre-filtering by metadata (document source, date) reduces search space.
- HITL escalations handled asynchronously; user receives immediate acknowledgement.

This HLD is a living document and should be reviewed against evolving infrastructure and LLM capabilities. All architecture decisions are subject to change based on load-testing outcomes.